

4.2. Componentes básicos de una interfaz

Contenido

1. Texto	1
2. Imágenes	2
3. Listas.....	2
4. Tablas	3
5. Etiquetas	4
6. Botones.....	4
7. Entrada de texto	5
7.1. Áreas de texto	7
8. Checkbox	7
9. Radio	9
10. Select.....	9
11. Slider	10
12. Formularios.....	11
12.1. Validación de formularios.....	12
13. Referencias.....	13

1. Texto

El texto es el elemento más común a la hora de presentar información en la interfaz.

Componentes:

- `<p>` : Párrafo
- `<h1>` , `<h2>` , `<h3>` , `<h4>` , `<h5>` , `<h6>` : Títulos
- `<span>` : Texto en línea

Ejemplo:

```
<h2>Esto es un título</h2>
<p>Esto es un párrafo que contiene un <span class="text-info">texto en
línea</span> con el color cambiado.</p>
```

Resultado:

Esto es un

Esto es un párrafo que contiene un **texto en línea** con el color cambiado.

Elementos de bloque y en línea

Los párrafos y títulos son elementos de bloque, es decir, ocupan todo el ancho de la página. El texto en línea se muestra en la misma línea que el texto anterior y posterior.

Aunque este comportamiento se puede modificar con CSS.

2. Imágenes

Las imágenes se insertan con la etiqueta `<img>`. Esta etiqueta no tiene contenido, solo atributos.

Ejemplo:

```

```

3. Listas

Las listas se crean con las etiquetas `<ul>` y `<ol>` para listas no ordenadas y ordenadas respectivamente. Cada elemento de la lista se crea con la etiqueta `<li>`.

Ejemplo:

```
<ul>
  <li>Elemento 1</li>
  <li>Elemento 2</li>
  <li>Elemento 3</li>
</ul>
<ol>
  <li>Elemento 1</li>
  <li>Elemento 2</li>
  <li>Elemento 3</li>
</ol>
```

Resultado:

- Elemento 1
 - Elemento 2
 - Elemento 3
-
1. Elemento 1
 2. Elemento 2
 3. Elemento 3

4. Tablas

Las tablas se crean con la etiqueta `<table>`. Cada fila de la tabla se crea con la etiqueta `<tr>` a celda con la etiqueta `<td>`. Para crear una cabecera de tabla se utiliza la etiqueta `<th>`.

Ejemplo:

```
<table class="table table-striped">
  <thead>
    <tr>
      <th scope="col">Nombre</th>
      <th scope="col">Apellidos</th>
      <th scope="col">Edad</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Nombre 1</td>
      <td>Apellidos 1</td>
      <td>Edad 1</td>
    </tr>
    <tr>
      <td>Nombre 2</td>
      <td>Apellidos 2</td>
      <td>Edad 2</td>
    </tr>
    <tr>
      <td>Nombre 3</td>
      <td>Apellidos 3</td>
      <td>Edad 3</td>
    </tr>
  </tbody>
</table>
```

Resultado:

Nombre	Apellidos	Edad
Nombre 1	Apellidos 1	Edad 1
Nombre 2	Apellidos 2	Edad 2
Nombre 3	Apellidos 3	Edad 3

5. Etiquetas

Las etiquetas son elementos que permiten resaltar texto. Se crean con la etiqueta `<mark>`.

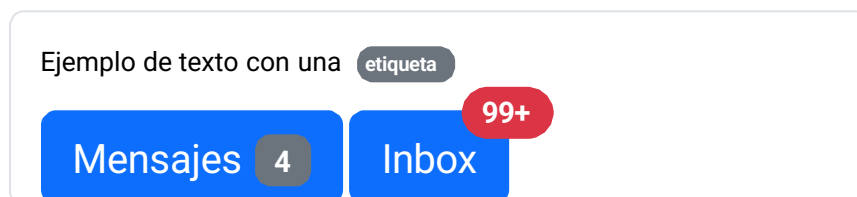
Ejemplo:

```
<p>Ejemplo de texto con una <span class="badge bg-secondary">etiqueta</span></p>

<button type="button" class="btn btn-primary">
  Mensajes <span class="badge text-bg-secondary">4</span>
</button>

<button type="button" class="btn btn-primary position-relative">
  Inbox
  <span class="position-absolute top-100 start-100 translate-middle badge
rounded-pill bg-danger">
    99+
    <span class="visually-hidden">unread messages</span>
  </span>
</button>
```

Resultado:



6. Botones

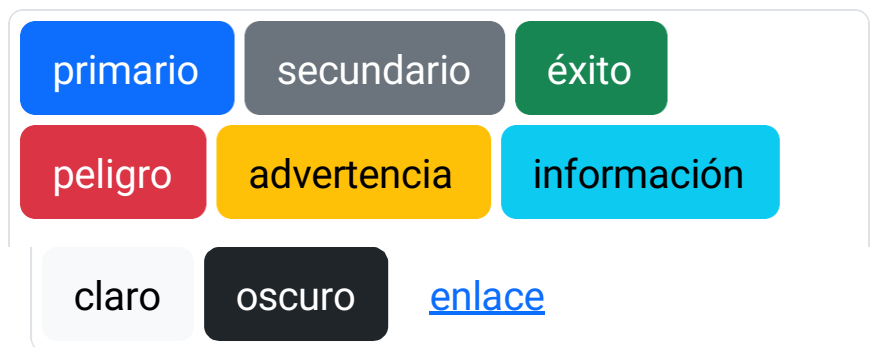
Los botones son elementos básicos que permiten al usuario interactuar con la aplicación.

Se crean con la etiqueta `<button>`

Ejemplo:

```
<button class="btn btn-primary">primario</button>
<button class="btn btn-secondary">secundario</button>
<button class="btn btn-success">éxito</button>
<button class="btn btn-danger">peligro</button>
<button class="btn btn-warning">advertencia</button>
<button class="btn btn-info">información</button>
<button class="btn btn-light">claro</button>
<button class="btn btn-dark">oscuro</button>
<button class="btn btn-link">enlace</button>
```

Resultado:



Para dar interacción a los botones debemos asociarles un evento. Esto se puede hacer con método `addEventListener()`

Ejemplo:

```
<button id="boton" class="btn btn-primary">primario</button>
```

```
const button = document.getElementById('boton');
button.addEventListener('click', () => {
  console.log('click');
});
```

7. Entrada de texto

La etiqueta `<input>` permite al usuario introducir texto en la aplicación, suele ir acompañada de una etiqueta `<label>` que indica el propósito del campo.

Ejemplo:

```
<label for="user" class="col-form-label">Usuario:</label>
<input type="text" class="form-control" id="user" value="usuario1">
<label for="password" class="col-form-label">Contraseña:</label>
<input type="password" class="form-control" id="password" value="no visible">
<label for="email" class="col-form-label">Email:</label>
<input type="email" class="form-control" id="email"
value="usuario1@domain.com">
```

Usuario:

usuario1

Contraseña:

.....

Email:

usuario1@domain.com

Una práctica muy común es el uso de etiquetas flotantes para indicar el propósito del campo. Esto se puede conseguir con la clase `form-floating` y el atributo `placeholder`.

Ejemplo:

```
<div class="form-floating mb-3">
  <input type="email" class="form-control" id="floatingInput" placeholder="
```

Usuario:

usuario1

Password:

Email:

Para leer o establecer el valor de un campo de texto se utiliza la propiedad `value`.

```
const user = document.getElementById('user');  
let usuario = user.value;  
user.value = 'nuevo nombre de usuario'
```

7.1. Áreas de texto

Las áreas de texto son similares a los campos de texto, pero permiten introducir texto en varias líneas. Cuando el texto no cabe en el área, se muestra una barra de desplazamiento vertical.

Ejemplo:

```
<label for="demo">Área de texto:</label>  
<textarea class="form-control" id="demo" rows="3">Texto de ejemplo.  
  Las áreas de texto permiten introducir textos más largos  
  y presentarlos en varias líneas.</textarea>
```

Área de texto:

Texto de ejemplo. Las áreas de texto permiten introducir textos más largos y presentarlos en varias líneas.

8. Checkbox

Las casillas de verificación permiten al usuario seleccionar una o varias opciones de una lista.

- Se crean con la etiqueta `<input>` y el atributo `type="checkbox"`.
- Para seleccionar una casilla por defecto se utiliza el atributo `checked`
 - Los atributos `name` y `value` permiten identificar el valor de la casilla seleccionada.

Ejemplo:

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" id="opcion1" name="opcion1"
value="op1">
  <label class="form-check-label" for="opcion1">
    Primera opción
  </label>
</div>
<div class="form-check">
  <input class="form-check-input" type="checkbox" id="opcion2" name="opcion2"
value="op2" checked>
  <label class="form-check-label" for="opcion2">
    Segunda opción
  </label>
</div>
```

☐ Primera opción

☒ Segunda opción

Alternativamente se pueden representar con interruptores (switch):

```
<div class="form-check form-switch">
  <input class="form-check-input" type="checkbox" role="switch" id="opcion1"
name="opcion1" value="op1">
  <label class="form-check-label" for="opcion1">
    Primera opción
  </label>
</div>
<div class="form-check form-switch">
```

```
  <input class="form-check-input" type="checkbox" role="switch" id="opcion2"
name="opcion2" value="op2" checked>
  <label class="form-check-label" for="opcion2">
    Segunda opción
  </label>
</div>
```

☐ Primera opción

☒ Segunda opción

Para leer o establecer el valor de una casilla de verificación se utiliza la propiedad `checked`.

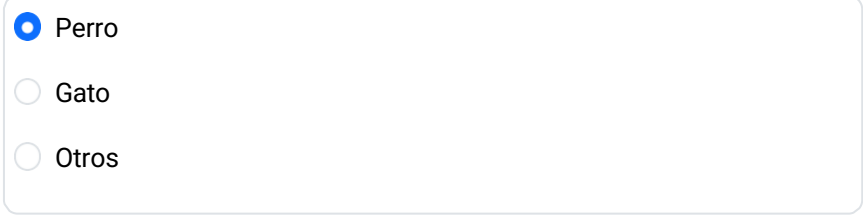
```
const opcion1 = document.getElementById('opcion1');
let valor = opcion1.checked;
if (!valor.checked)
  opcion1.checked = true;
```

9. Radio

Los botones de radio permiten al usuario seleccionar sólo una opción de una lista.

- Se crean con la etiqueta `<input>` y el atributo `type="radio"`.
- Para seleccionar una opción por defecto se utiliza el atributo `checked`.
- Para agrupar los botones de radio se utiliza el atributo `name` con el mismo valor.
- El atributo `value` permite identificar el valor de la opción seleccionada.

```
<div class="form-check">
  <input class="form-check-input" type="radio" id="mascota-perro"
name="mascota" value="perro" checked>
  <label class="form-check-label" for="mascota-perro">
    Perro
  </label>
</div>
<div class="form-check">
  <input class="form-check-input" type="radio" id="mascota-gato"
name="mascota" value="gato">
  <label class="form-check-label" for="mascota-gato">
    Gato
  </label>
</div>
<div class="form-check">
  <input class="form-check-input" type="radio" id="mascota-otros"
name="mascota" value="otros">
  <label class="form-check-label" for="mascota-otros">
    Otros
  </label>
</div>
```



A form with three radio buttons. The first button, labeled 'Perro', is selected and has a blue dot. The second button, labeled 'Gato', is unselected. The third button, labeled 'Otros', is unselected.

Para leer o establecer el valor de un botón de radio se utiliza la propiedad `checked`.

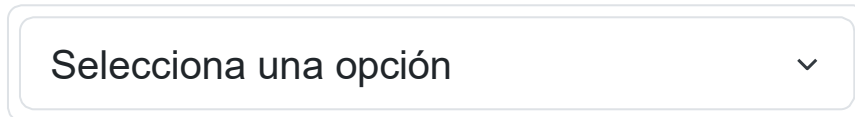
```
const perro = document.getElementById('mascota-perro');
let valor = perro.checked;
if (!valor.checked)
  perro.checked = true;
```

10. Select

Los selectores permiten al usuario seleccionar una opción de una lista desplegable.

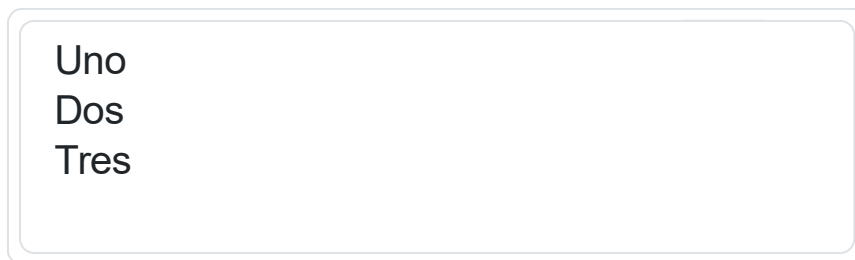
- Se crean con la etiqueta `<select>`.
- El atributo `name` permite identificar el valor de la opción seleccionada.

```
<select class="form-select" name="opcion">
  <option selected>Selecciona una opción</option>
  <option value="1">Uno</option>
  <option value="2">Dos</option>
  <option value="3">Tres</option>
</select>
```



También se pueden crear selectores múltiples, que permiten al usuario seleccionar varias opciones de una lista desplegable. Para ello se utiliza el atributo `multiple`.

```
<select class="form-select" name="opcion" multiple>
  <option value="1">Uno</option>
  <option value="2">Dos</option>
  <option value="3">Tres</option>
</select>
```



Para leer o establecer el valor de un selector se utiliza la propiedad `value`.

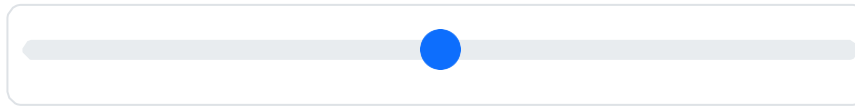
```
const selector = document.getElementById('selector');
let valor = selector.value;
if (selector.value !== 2)
  selector.value = 2
```

11. Slider

Los selectores permiten al usuario seleccionar un valor de una barra deslizante.

- Se crean con la etiqueta `<input>` y el atributo `type="range"`.
 - El atributo `name` permite identificar el valor seleccionado.
 - Se puede especificar el valor mínimo y máximo con los atributos `min` y `max`.
- El atributo `step` permite especificar el incremento del valor.

```
<input class="form-range" type="range" min="0" max="100" step="1"
name="slider">
```



Para leer o establecer el valor de un selector se utiliza la propiedad `value`.

```
const slider = document.getElementById('slider');
let valor = slider.value;
if (slider.value < 50)
    slider.value = 50
```

12. Formularios

Los formularios son elementos que permiten al usuario introducir datos en la aplicación.

Se crean con la etiqueta `<form>`. Los campos de texto, áreas de texto y botones se incluyen dentro del formulario.

Podemos añadir el atributo `required` a los campos de texto para indicar que son obligatorios.

Ejemplo:

```
```html
<form>
 <div class="mb-3">
 <label for="user" class="col-form-label">Usuario:</label>
 <input type="text" class="form-control" id="user" value="usuario1"
 required>
 </div>
 <div class="mb-3">
 <label for="password" class="col-form-label">Contraseña:</label>
 <input type="password" class="form-control" id="password" value="no
visible">
 </div>
 <div class="mb-3">
 <label for="email" class="col-form-label">Email:</label>
 <input type="email" class="form-control" id="email" value="">
 </div>
 <div class="mb-3">
 <label for="demo">Área de texto:</label>
 <textarea class="form-control" id="demo" rows="3">Texto de
ejemplo. Las áreas de texto permiten introducir textos más largos y
presentarlos en varias líneas.</textarea>
 </div>
 <button type="submit" class="btn btn-primary">Enviar</button>
</form>
```

Nombre

Apellidos

Usuario

Contraseña

Email:

☐ Default radio

☒ Default checked radio

☐ Acepto las condiciones

Enviar

Restablecer

## 12.1. Validación de formularios

Los formularios se pueden validar con JavaScript. Para ello se utiliza el método

`checkValidity()` del formulario. *Ejemplo*

```
const form = document.querySelector('form');
form.addEventListener('submit', (event) => {
 if (!form.checkValidity()) {
 event.preventDefault();
 event.stopPropagation();
 }
 form.classList.add('was-validated');
});
```

- El método `checkValidity()` devuelve `true` si el formulario es válido y `false` en caso contrario.
- El método `preventDefault()` evita que se envíe el formulario.
- El método `stopPropagation()` evita que se propague el evento.
- La clase `was-validated` permite mostrar los mensajes de error.
- Los mensajes de error se muestran con la clase `invalid-feedback`.

## 13. Referencias

- [Bootstrap](#)
- [Electron](#)
- [W3Schools](#)
- [MDN Web Docs](#)